SpecChain — Part 1: Algebraic Foundations, Complexity, and Mechanization (Normative Revision)

SpecChain Working Group

September 17, 2025

Abstract

This revision makes *Part 1* a normative, machine-checkable contract. We (i) pin the run fingerprint `proof_surface` to a byte-precise canonicalization rule (`canon/v1`) so that any auditor can *recompute the exact bytes* bound by BLAKE3-256; (ii) fix global naming (`proof_surface`, `root_print`) and deprecate obsolete terms; (iii) resolve "unknowns" semantics by treating per-axis unknown as an *absorbing bottom* $\bot$ (fail-closed) in $\mathbf{Q}$ with strict ledgering; (iv) upgrade the acceptance algebra to a policy-driven typed quantale/quantaloid and expose an uncertainty-aware lift $\mathbf{Q}'$; (v) unify computational hardness via a single strong NP-hardness statement for exact frontier debit, and (vi) state sound upper-bounding regimes with mechanization hooks. A Lean/Coq extraction surface provides verified checkers (frontier guards, subadditivity spot checks, and `proof_surface` binding). Later parts carry domain ontologies, the Normative Loader and runtime assurance, governance, and end-to-end safety cases (Part 2–Part 5).

1 Global Objects, Notation, and Thin Categories

Definition 1.1 (Posets as thin categories)**.** A poset $(P, \leq)$ induces a *thin category* P with objects the elements of P and a unique morphism $x \to y$ iff $x \leq y$. Identities are the reflexivity witnesses $x \leq x$; composition is inherited from transitivity. A map $f : (P, \leq) \to (Q, \preceq)$ is monotone iff it is a functor $\mathsf{P} \to \mathsf{Q}$.

Remark 1.2 (Functorial feasibility and schedule composition)**.** Precedence/feasibility constraints are modeled as functors between thin categories; this lets us compose local orders into global ones. In particular, evaluation orders (defined in Section 9) are functorially composed from stage-local constraints, ensuring that any functorial image of a feasible schedule remains feasible.

Proposition 1.3 (Products and embeddings are functorial)**.** *If $(P_i, \leq_i)$ are posets, the product poset $\prod_i P_i$ with coordinatewise order induces the categorical product $\prod_i \mathsf{P_i}$ in Poset. Projections $\pi_j : \prod_i P_i \to P_j$ and diagonals $\Delta : P \to P \times P$ are monotone and hence functorial.*

Remark 1.4 (Order-enriched lens)**.** Throughout we freely view monotone maps both as set-theoretic functions and as morphisms in thin categories. This "order-enriched" lens will be used for algebraic adapters (homomorphisms into $(\mathbf{Q}, \wedge)$) and for policy refinement maps (Section 10), where functoriality certifies that strengthening policies preserves adapter monotonicity.

Notation hygiene (global conventions). These are *normative* naming and symbol choices used across all parts:

- **Run fingerprint.** The canonical field name is `proof_surface` (a JSON key spelled `"proof_surface"`). The obsolete term "`proof_print`" is forbidden in new outputs.
- **Genesis constant.** `root_print` denotes the fixed digest for the genesis run.
- **Terminal stage.** We write Ω (capital Omega) for the terminal adapter stage; the symbol ω is never used for this purpose.
- **Unknowns.** Unless stated otherwise, each axis carries an absorbing bottom element $\bot$ that *fails closed* under aggregation in $\mathbf{Q}$ (see Section 2); user-facing variants (e.g., UI "unknown") must normalize to $\bot$ before entry into the algebra.

Example 1.5 (Tiny feasibility sketch)**.** Let P be the poset of EF/DRO/PR/CBF/Ω milestones ordered by "must-occur-before" and let Q refine P by inserting additional adapter micro-steps. The inclusion $i : \mathsf{P} \to \mathsf{Q}$ is a functor; composing any feasible schedule $s : \mathbf{1} \to \mathsf{P}$ with i yields a feasible schedule in Q. This underlies the refinement of the high-level order in Section 9 by implementation-level orchestrations.

2 Acceptance Algebra Q: Product With Absorbers

Definition 2.1 (Axis-typed carriers with identity and absorber)**.** Fix disjoint index sets S (sum-axes) and M (max-axes), with $I = S \cup M$. For each $i \in I$ let $(Q_i, \leq_i)$ be a poset equipped with:

(i) an *identity* element e_i (for S: $e_i = 0_i$; for M: $e_i = -\infty_i$);

(ii) an *absorbing bottom* $\bot_i$ such that $\bot_i \leq_i x$ for all $x \in Q_i$ and the axis aggregation (defined below) satisfies $x \otimes_i \bot_i = \bot_i \otimes_i x = \bot_i$.

The per-axis aggregation law is

$$x \otimes_i y \;=\; \begin{cases} x + y, & i \in S \quad \text{(commutative monoid)} \\ \max\{x, y\}, & i \in M \quad \text{(idempotent join)} \end{cases}$$

and is required to be monotone in both arguments w.r.t. $\leq_i$.

Definition 2.2 (Product order and coordinatewise aggregation)**.** Define the acceptance algebra

$$\mathbf{Q} \;\coloneqq\; \prod_{i \in I} Q_i, \qquad q \leq_{\mathbf{Q}} q' \iff \forall i \in I,\ q_i \leq_i q'_i.$$

The global aggregation $\wedge$ is taken coordinatewise:

$$(q \wedge q')_i \;=\; \begin{cases} q_i + q'_i, & i \in S \text{ (sum-axis)} \\ \max\{q_i, q'_i\}, & i \in M \text{ (max-axis)} \end{cases},$$

with the absorber and identity inherited per axis from Definition 2.1. Write $\mathbf{e} \in \mathbf{Q}$ for the vector of identities $(e_i)_i$ and $\bot^{(J)} \in \mathbf{Q}$ for the vector with $(\bot^{(J)})_i = \bot_i$ if $i \in J$ and e_i otherwise.

Proposition 2.3 (Algebraic laws for $(\mathbf{Q}, \wedge)$)**.** *For all $q, r, s \in \mathbf{Q}$:*

(a) ***Monotonicity:*** *if $q \leq_{\mathbf{Q}} r$ and $s \leq_{\mathbf{Q}} t$, then $q \wedge s \leq_{\mathbf{Q}} r \wedge t$.*

(b) ***Associativity/commutativity:*** *$q \wedge (r \wedge s) = (q \wedge r) \wedge s$ and $q \wedge r = r \wedge q$.*

(c) ***Axiswise idempotence:*** *$(q \wedge q)_i = q_i$ iff $i \in M$.*

(d) ***Identity:*** $q \wedge \mathbf{e} = q$.

(e) ***Absorbers ledgered:*** $q \wedge \bot^{(J)}$ *equals* q *on* $I \setminus J$ *and* $\bot$ *on* J*; in particular, no coordinate can "cancel" an* $\bot$.

Remark 2.4 (Policy note: unknowns are fail-closed)**.** The element $\bot_i$ is *both* an order-bottom and an aggregation absorber on axis i. This choice is normative: any appearance of unknown/missing/invalid evidence *must* descend to $\bot$ and persist through aggregation. User- or UI-level sentinels (e.g., a display "unknown") are required to be normalized to $\bot$ before entering $\mathbf{Q}$; see also the cross-cutting invariant on strict $\bot$ ledgering (Section 15).

Example 2.5 (Two-axis intuition and $\bot$ propagation)**.** Let energy be a sum-axis (J) and disclosure risk a max-axis (dimensionless). If $q = (50\text{ J}, 0.2)$ and $q' = (20\text{ J}, \bot)$ then

$$q \wedge q' \;=\; (70\text{ J}, \bot).$$

Energy sums; the risk coordinate is absorbed by $\bot$ and remains unknown, ensuring fail-closed behavior on that axis.

Remark 2.6 (Backward compatibility and upgrade path)**.** The classic (S, M) design is recovered by taking $\otimes_i = +$ on S and $\otimes_i = \max$ on M, with identities (0_i) and $(-\infty_i)$ as usual and with a distinguished absorber $\bot_i$. Quantale/quantaloid structure and policy-driven refinements are provided in Section 10; uncertainty carriers and acceptance lifting appear in Section 11.

Proposition 2.7 (Adapter homomorphisms into $(\mathbf{Q}, \wedge)$)**.** *If* A *is an adapter that (i) is axis-isolating (writes only to a declared subset* $J \subseteq I$*), (ii) is monotone on its input poset, and (iii) aggregates independent inputs by the underlying per-axis laws, then* A *is a* $(\mathbf{Q}, \wedge)$*-homomorphism:* $A(x \cup y) = A(x) \wedge A(y)$ *(where defined) and* $x \preceq x' \Rightarrow A(x) \leq_{\mathbf{Q}} A(x')$*. Axis isolation is enforced at runtime by the Normative Loader (see Sections 7 and 15).*

3 Typed Effects Σ and the Retune Preorder

Definition 3.1 (Effects, rank, and non-expansive retunes)**.** Let Σ be a finite multiset of *typed obligations* (each element carries a type tag and optional metadata). Fix a *rank map*

$$\mu : \Sigma \longrightarrow \mathbb{N} \times \mathbb{N}, \qquad \mu(\Sigma) \;=\; (r_0(\Sigma),\, r_2(\Sigma)),$$

ordered lexicographically. Intuitively, r_0 counts baseline invariant debt (e.g., currently failing mandatory checks), while r_2 counts outstanding adapter obligations (e.g., pending certificates or budgets). Define the *retune preorder*

$$\Sigma' \;\preceq\; \Sigma \quad\Longleftrightarrow\quad \mu(\Sigma') \;\leq_{\text{lex}}\; \mu(\Sigma).$$

A *retune* $\Sigma \rightsquigarrow \Sigma'$ is *legal* iff it is *non-expansive*: $\Sigma' \preceq \Sigma$. Every legal retune must be recorded by a byte-level `RetuneCert`.

Assumption 3.2 (Obligation non-expansion)**.** All system-initiated retunes satisfy Definition 3.1. External proposals that violate non-expansion are rejected fail-closed with a verifiable `RejectionCert`. Both certificates are canonicalized under `canon/v1` and included in the evidence bundle at Ω (hence bound by `proof_surface`).

Proposition 3.3 (Preorder properties and progress)**.** *The relation $\preceq$ is a reflexive, transitive preorder. Moreover, if a run employs only legal retunes, then the rank sequence $\mu(\Sigma_0) \succeq_{\text{lex}} \mu(\Sigma_1) \succeq_{\text{lex}} \cdots$ is monotonically nonincreasing and well-founded; any infinite strictly decreasing chain is impossible. Consequently, any retune loop must eventually stutter (become rank-stable), at which point further legal state changes cannot increase r_0 nor r_2.*

Remark 3.4 (Operational alignment with the Loader)**.** The Normative Loader (Part 3) maintains an internal rank that is *definitionally* μ. Stage **E5** (CBF; see Section 9) is the only stage at which retunes are permitted. Axis isolation implies that retunes affect obligation state Σ without writing to undeclared axes of $\mathbf{Q}$ or $\mathbf{Q}'$; any attempt to do so violates I6 and yields a `RejectionCert`.

Proposition 3.5 (Compatibility with acceptance algebra)**.** *Let A be any axis-isolating, monotone adapter (Definition 2.7). If $\Sigma \rightsquigarrow \Sigma'$ is a legal retune then, holding inputs fixed, the adapter outputs in $(\mathbf{Q}, \wedge)$ satisfy*

$$A[\Sigma'] \leq_{\mathbf{Q}} A[\Sigma] \quad \textit{on all axes whose pricing depends (monotonically) on the retuned obligations.}$$

In particular, a retune cannot improve *an acceptance coordinate unless it* reduces *the corresponding obligation rank component.*

Certificate surface (normative, `canon/v1`). A `RetuneCert` is a canonical JSON object with at least:

- `"kind"`: `"RetuneCert"`,
- `"from_rank"`: $[r_0(\Sigma), r_2(\Sigma)]$, `"to_rank"`: $[r_0(\Sigma'), r_2(\Sigma')]$,
- `"delta"`: a canonicalized multiset diff (removals/additions),
- `"witness_hash"`: content digest of the predicate/evidence that justified the retune,
- `"time"`: monotone logical clock (run-local), and `"stage"`: `"CBF"`.

Its bytes are produced by encoding the JSON under `canon/v1` and are included in the Ω evidence bundle. A `RejectionCert` mirrors this surface with `"kind"`: `"RejectionCert"` and a `"predicate_id"` naming the violated invariant (e.g., `"I6_AXIS_ISOLATION"`).

Listing 1: Illustrative `RetuneCert` (canon/v1 shape).

```
{"delta":{"add":["cbf:budget:throttle"],"rem":["pr:need:ThetaCert"]},
 "from_rank":[1,3],
 "kind":"RetuneCert",
 "stage":"CBF",
 "time":42,
 "to_rank":[0,2],
 "witness_hash":"b3b7f1..."}
```

Example 3.6 (Tiny retune)**.** Suppose $\mu(\Sigma) = (1, 3)$: one failing baseline check (r_0=1) and three outstanding obligations (r_2=3). A certified PR proof discharges one baseline and one obligation, yielding $\mu(\Sigma') = (0, 2)$. The retune is legal since $(0, 2) \leq_{\text{lex}} (1, 3)$; a `RetuneCert` with the diff and witness hash is emitted and later bound into `proof_surface`.

Remark 3.7 (Determinism and $\perp$)**.** Obligation fields that are *unknown* at CBF must be represented by the absorber $\perp$ on their priced axes in $\mathbf{Q}$ (or by an explicit missing tag in Σ that *induces* $\perp$ when priced). Any attempt to retune while masking $\perp$ is rejected. Thus, retunes cannot "launder" unknowns.

Table 1: Block header fields (normative). All JSON is encoded under `canon/v1`.

Field	Type	Constraint (canonical)
`parent_hashes`	List[Digest]	Distinct digests, lexicographically sorted; references only *finalized* parents.
`suite`	FixedID	Protocol identifier frozen for the run (e.g., `"specchain/v1"`).
`meta`	KV	Flat key/value map for non-consensus metadata; keys unique, ASCII; values JSON-scalars.

4 Ledger: Content-Addressed DAG of Blocks

Definition 4.1 (Digests and canonical bytes)**.** Let *Digest* be a 32-byte output of BLAKE3-256. Any JSON artifact in this specification is serialized to bytes under the canonicalization contract `canon/v1` (UTF-8, sorted keys, no whitespace, finite IEEE-754 numbers in minimal decimal form; strings escaped per JSON). We write $X_{\texttt{canon/v1}}$ for the bytes of a JSON value X encoded by `canon/v1`.

Definition 4.2 (Block, hash, and integrity)**.** A block is a pair $(\mathit{header}, \mathit{payload})$ where:

- *header* is a JSON object with fields of Table 1; its bytes are $\mathit{hbytes} := \mathit{header}_{\texttt{canon/v1}}$.
- *payload* is a canonical JSON object carrying the evidence bundle for the commit (including, at Ω, the `proof_surface`); its bytes are $\mathit{pbytes} := \mathit{payload}_{\texttt{canon/v1}}$.

The *content address* is

$$\mathsf{H}(b) \;:=\; \text{BLAKE3-256}(\mathit{hbytes} \,\|\, \mathit{pbytes}).$$

Genesis has `parent_hashes` $= [\,]$. The set of blocks, with directed edges from $\mathsf{H}(b)$ to each digest in `parent_hashes`, forms a graph Ledger.

Assumption 4.3 (Evaluation/commit acyclicity)**.** Commits may reference only *finalized* parents; `parent_hashes` contains no duplicates and no self-references. Under collision resistance of BLAKE3-256, the induced graph Ledger is a DAG. (Seed discipline and the evaluation order establishing causality are given in Section 9 and Definition 8.4.)

Proposition 4.4 (Canonical re-encoding is stable)**.** *Let $b = (\mathit{header}, \mathit{payload})$ be a block. Any recomputation of hbytes and pbytes from the same JSON values using* ***canon/v1*** *yields the* identical *byte strings; hence $\mathsf{H}(b)$ is replay-stable. In particular, permitted transformations of pretty-printed JSON (whitespace, key order, numeric formats, NaN/Inf) are* not *allowed under* ***canon/v1*** *and would alter bytes, so they are rejected upstream by the Loader.*

Remark 4.5 (Deterministic inclusion of artifacts)**.** At Ω, the *payload must* include the run's `proof_surface` (see Definition 5.1) and the byte-exact evidence bundle to which it refers. Because both header and payload are encoded under `canon/v1`, the block's hash binds the precise artifact set; any deviation is detectable by byte-level recomputation.

5 Run Fingerprints: `root_print` and `proof_surface`

Definition 5.1 (Genesis constant and run fingerprint)**.** Let $H =$ BLAKE3-256 and let `canon/v1` be the canonicalization contract of Definition 4.1. Define the *run fingerprint* `proof_surface` as the digest of a canonical JSON object SurfaceV1 with required keys:

Key	Type	Description (canonical under `canon/v1`)
`"raw_data_digest"`	Digest	Digest of the canonical raw input bytes (EF).
`"envlock_id"`	FixedID	Identifier of the deterministic envelope (`EnvLock`,`ReplayPlan`).
`"config_cert"`	Digest	Digest of the canonicalized configuration/policy certificate for the run.
`"adapter_decls"`	Digest	Digest of the canonical adapter declarations/versions in effect.
`"axis_types"`	Digest	Digest of the serialized axis policy catalog (see Section 10).
`"evidence_bundle"`	Digest	Digest (e.g., Merkle root) of the byte-exact evidence set closed at Ω.
`"access_policy"`	Digest	Digest of access-control policy binding (keys, roles, redaction rules).
`"audit_plan"`	Digest	Digest of the audit plan/queries to be satisfied on replay.

Formally,

$$\texttt{proof_surface} \coloneqq H\Big(\mathsf{SurfaceV1}_{\texttt{canon/v1}}\Big).$$

The *genesis constant* `root_print` is the fixed digest H of the genesis run's SurfaceV1 (published with the suite definition). The JSON field name for the fingerprint is `"proof_surface"`; the obsolete term `"proof_print"` is forbidden in new outputs.

Remark 5.2 (Canon-bytes and scope of binding)**.** Only the *bytes* of SurfaceV1 under `canon/v1` are hashed. Each listed key points (by digest) to separately canonicalized artifacts that are included in the block payload at Ω (Definition 4.2). Thus, `proof_surface` binds (i) the inputs/envelope/config, (ii) the axis policy surface used to interpret $\mathbf{Q}/\mathbf{Q}'$, and (iii) the complete evidence set and governance context required for replay and audit.

Proposition 5.3 (Re-encoding equality and rejection)**.** *Let S be a JSON value that meets the schema of Definition 5.1. Any auditor who recomputes $S_{\texttt{canon/v1}}$ obtains bytewise equality with the producer; therefore the recomputed $H(\cdot)$ equals `proof_surface`. Inputs that violate `canon/v1` (unsorted keys, whitespace, non-finite numbers, non-minimal numeric forms) are rejected upstream; no producer may "normalize" such inputs post hoc without changing bytes.*

Listing 2: Illustrative SurfaceV1 object (canon/v1 shape).

```
{"access_policy":"d9f4b2...",
 "adapter_decls":"ab77c4...",
 "audit_plan":"f02c6e...",
 "axis_types":"41a0da...",
 "config_cert":"3b11ff...",
 "envlock_id":"EnvLock/replayplan-v3",
 "evidence_bundle":"7d88ef...",
 "raw_data_digest":"2f0a9c..."}
```

Remark 5.4 (Seed discipline and acyclicity)**.** PRNG seeds derive from (`root_print`, `suite`, `parent_hashes`, `proof_surface`) inside the deterministic envelope; see Definition 8.4. Because `proof_surface` is finalized at Ω before commit, seeds cannot depend on yet-to-be-committed hashes, ruling out causal cycles.

6 Frontier Debit: Rows, Guards, and Axioms

Definition 6.1 (Frontier rows and debit)**.** A *frontier row* is a dimensionless record of a priced crossing

$$\langle\ \mathit{frontier_id} : \mathsf{FixedID},\ \mathit{op_type} : \mathsf{Enum},\ dF/\nu : \mathbb{R}_{\geq 0},\ \widehat{\nu}_{\min} : \mathbb{R}_{>0},\ c\Sigma : \mathsf{Digest},\ \kappa : \mathbb{R}_{\geq 0}\ \rangle.$$

Given a policy π that declares which axes are priced and how (Section 10), let $\mathsf{price}_\pi : \mathsf{Row} \to \mathbf{Q}$ be the per-row axis-isolating, monotone pricing map, and extend it to a multiset L of rows by coordinatewise aggregation in $(\mathbf{Q}, \wedge)$:

$$\Delta_{\mathrm{fr}}(L, \pi)\ :=\ {}_{r \in L}\mathsf{price}_\pi(r).$$

When the policy is clear we write $\Delta_{\mathrm{fr}}(L)$.

Assumption 6.2 (Frontier safety guard (normative))**.** Each row must satisfy the guard inequality $\kappa \leq C_{\mathrm{salt}}\,(1 + C_I)$ with $C_{\mathrm{salt}} > 0$ and $C_I \geq 0$ dimensionless, derived from declared constants in the policy context. Any row for which the priced inputs on a priced axis are $\bot$ *forces* the guard to fail (deterministic rejection with a `RejectionCert`); the Loader MUST NOT accept a row that would merely map to $\bot$ *post hoc*.

Proposition 6.3 (Minimal algebraic interface (axioms for Part 1))**.** *For any legal log L and policy π:*

(F1) ***Identity.*** $\Delta_{\mathrm{fr}}(\varnothing) = \mathbf{e}$*, the per-axis identity in* $\mathbf{Q}$*.*

(F2) ***Nonnegativity.*** *On sum-axes $i \in S$, $\big(\Delta_{\mathrm{fr}}(L)\big)_i \geq 0_i$; on max-axes $i \in M$, $\big(\Delta_{\mathrm{fr}}(L)\big)_i \geq -\infty_i$.*

(F3) ***Subadditivity (concat).*** *For compatible sequences L_1, L_2,*

$$\Delta_{\mathrm{fr}}(L_1 \uplus L_2)\ \leq_{\mathbf{Q}}\ \Delta_{\mathrm{fr}}(L_1) \wedge \Delta_{\mathrm{fr}}(L_2).$$

(On sum-axes this is the usual $\leq$ with $+$; on max-axes it is idempotent join.)

(F4) ***Policy monotonicity.*** *If $\pi \preceq \pi'$ (policy refinement; cf. Section 10), then $\Delta_{\mathrm{fr}}(L, \pi) \leq_{\mathbf{Q}} \Delta_{\mathrm{fr}}(L, \pi')$.*

(F5) ***Dimensionless invariance.*** *Δ_{fr} is invariant under any consistent rescaling of physical units in evidence sources; pricing kernels normalize to dimensionless coordinates.*

These are axioms *of Part 1; formal constructions/proofs appear in Part 2.*

Remark 6.4 (Unknowns and fail-closed pricing)**.** If any priced ingredient of a row is $\bot$ on an axis i, then either (a) the row is rejected at guard time per Definition 6.2, or (b) that axis of $\mathsf{price}_\pi(r)$ is $\bot_i$ and thus absorbs under $\wedge$ by Definition 2.3. No subsequent aggregation may "cancel" an $\bot$.

Example 6.5 (Toy log and concatenation)**.** Let L_1 contain one row with energy (sum-axis) cost 5 and risk (max-axis) 0.2; let L_2 contain one row with energy 7 and risk $\bot$. Then

$$\Delta_{\mathrm{fr}}(L_1) = (5,\ 0.2), \qquad \Delta_{\mathrm{fr}}(L_2) = (7,\ \bot), \qquad \Delta_{\mathrm{fr}}(L_1 \uplus L_2) = (12,\ \bot),$$

and Definition 6.3(F3) holds since $(12,\ \bot) \leq_{\mathbf{Q}} (5, 0.2) \wedge (7, \bot) = (12,\ \bot)$.

Mechanization hooks (extracted checkers). Part 1 exposes the following checker surfaces (see Section 14):

(C1) *Row guard validator:* verifies $\kappa \le C_{\mathrm{salt}}(1 + C_I)$ and rejects on any $\perp$ in priced inputs.

(C2) *Subadditivity spot-checker:* samples concatenations L_1, L_2 and refutes logs violating (F3) with a witnessed counterexample.

Both checkers operate on canon-bytes (`canon/v1`) and feed into `proof_surface` binding.

7 Adapter Pipeline and Normative Loader

Definition 7.1 (Stages and adapter interfaces)**.** The pipeline consists of the stages

$$\mathbf{EF} \;\to\; \mathbf{DRO} \;\to\; \mathbf{PR} \;\to\; \mathbf{CBF} \;\to\; \Omega$$

(*Evaluation Order* in Section 9). Each stage S exposes an *adapter* A_S with declared output axes $I_S \subseteq I$ and a total, monotone map into $(\mathbf{Q}, \wedge)$ that is *axis-isolating*:

$$A_S : \mathcal{X}_S \longrightarrow \mathbf{Q}, \qquad (A_S(x))_i = e_i \;\text{ for all } i \notin I_S.$$

Adapters aggregate independently provided inputs by the per-axis laws, so that (where defined) $A_S(x \cup y) = A_S(x) \wedge A_S(y)$ and $x \preceq x' \Rightarrow A_S(x) \le_{\mathbf{Q}} A_S(x')$ (cf. Definition 2.7).

Assumption 7.2 (Invariant I6: Axis isolation (normative))**.** For every stage S, writes are confined to the declared axes I_S. Any attempt to write outside I_S is a violation detected by the Loader and results in a `RejectionCert`.

Normative Loader (overview). The *Normative Loader* orchestrates the pipeline with fail-closed behavior and enforces the following invariants (non-exhaustive, but normative for Part 1):

I1. **I6** (Axis isolation). Enforced per Definition 7.2.

I2. **I7** (Canon-only IO). All JSON artifacts must be valid under `canon/v1`; non-canonical inputs are rejected upstream.

I3. **I8** (Deterministic envelope). Execution occurs inside (`EnvLock`, `ReplayPlan`); floating point, RNG, and serialization are pinned.

I4. **I9** (Guard-first pricing). *Before* any priced contribution is accepted from a frontier row, the guard of Definition 6.2 must pass; any $\perp$ in priced inputs triggers rejection (not silent absorption).

I5. **I10** (Retune non-expansion). Retunes may occur only at **CBF** and must satisfy Definition 3.1; a `RetuneCert` is emitted.

I6. **I11** (Seed discipline). PRNG seeds are derived from (`root_print`, `suite`, `parent_hashes`, `proof_surface`) inside `EnvLock` (see Definition 8.4).

I7. **I12** (Evidence closure). At Ω the evidence bundle is closed and digested; no late material changes are permitted.

I8. **I13** (`proof_surface` binding). The `proof_surface` is computed from the canonical SurfaceV1 (Definition 5.1) and inserted into the payload prior to commit.

I9. **I14** (Commit DAG hygiene). Commits reference only finalized parents; see Definition 4.3.

Remark 7.3 (No-bypass rule)**.** No global heuristic, cache, or out-of-band control flow may bypass frontier pricing, guards, budget enforcement, or Loader checks. Any such attempt must produce a `RejectionCert` and halt deterministically.

Proposition 7.4 (Loader-enforced homomorphism surface)**.** *Assuming **I6**–**I9**, every stage adapter A_S behaves as a $(\mathbf{Q}, \wedge)$-homomorphism on its declared axes and is monotone on inputs. In particular, the combined effect $A_{\mathrm{EF}} \wedge A_{\mathrm{DRO}} \wedge A_{\mathrm{PR}} \wedge A_{\mathrm{CBF}}$ is well-defined and independent of input grouping, with any violation detected by **I9**.*

Adapter declarations (normative, `canon/v1`). Each run includes a canonical *adapter declaration set* whose digest is the `"adapter_decls"` field of SurfaceV1 (Definition 5.1). The minimal JSON shape is:

- `"id"`: fixed identifier of the adapter instance;
- `"stage"`: one of `"EF"`, `"DRO"`, `"PR"`, `"CBF"`, `"Omega"`;
- `"axes"`: sorted list of declared axes I_S;
- `"version"`: semantic version or code digest;
- `"policy"`: digest of the axis-policy catalog this adapter targets.

Listing 3: Illustrative adapter declaration (canon/v1).

```
{"axes":["energy","risk"],
 "id":"adapter.dro.frontier-v2",
 "policy":"41a0da...",
 "stage":"DRO",
 "version":"2.3.1+sha.7c9e1a"}
```

Remark 7.5 (Interaction with Σ)**.** Stage **CBF** is the *only* locus where obligation state Σ changes are permitted, and then only via legal retunes (Definition 3.1). PR and earlier stages may *propose* obligations (as data) but cannot alter Σ without passing through **CBF**.

Remark 7.6 (Unknowns across the pipeline)**.** Unknown or invalid upstream evidence that would affect a priced axis must be represented as $\bot$ and handled per Definition 6.2. Silent propagation into $\mathbf{Q}$ is prohibited; either reject with a `RejectionCert` or produce an explicit $\bot$ that is then absorbed by $\wedge$ (Definition 2.3).

8 Deterministic Replay Envelope and Threat Model

Assumption 8.1 (Deterministic replay envelope)**.** All computations execute inside a fixed environment (`EnvLock`, `ReplayPlan`) that pins, at minimum:

- **Floating point**: IEEE-754 mode, rounding, denormals, fused-multiply-add, BLAS/LAPACK kernels and versions.
- **RNG**: algorithm, personalization string, seeding API and reseed policy; sampling libraries and distributions.
- **Serialization**: JSON (`canon/v1`), byte-order, text encoding (UTF-8), and any binary codecs used in evidence ingestion.

- **Time/OS**: run-local logical clocks for ordering; no wall-clock or nondeterministic OS entropy is permitted to influence priced outputs.
- **Code/proof**: content digests of executables, adapters, and proof scripts; any dynamic loading paths are frozen by digest.

Under this envelope, *all* stage outputs are bitwise-replayable from the bytes bound by `proof_surface` (Definition 5.1).

Assumption 8.2 (Threat model and fail-closed safety)**.** Standard cryptographic primitives (including BLAKE3-256) are assumed secure. Adapters may be adversarial subject to Loader enforcement. The ledger is public, append-only, and content-addressed (Section 4). Any violation of normative invariants (Section 7) yields a deterministic halt with a `RejectionCert` whose bytes are canonicalized under `canon/v1` and bound into the next commit attempt (or persisted off-ledger if no commit occurs).

Rejection certificate (normative, `canon/v1`). A `RejectionCert` is a canonical JSON object with at least:

- `"kind"`: `"RejectionCert"`,
- `"predicate_id"`: identifier of the violated invariant (e.g., `"I6_AXIS_ISOLATION"`, `"CANON_VIOLATION"`),
- `"failing_witness_hash"`: Digest of the artifact that demonstrated the failure,
- `"seed_snapshot"`: adapter-local RNG state snapshot (or its digest),
- `"parent_tip"`: latest finalized parent digest at the time of halt,
- `"time"`: run-local logical clock tick, `"stage"`: stage where the halt occurred.

Its bytes $\cdot_{\texttt{canon/v1}}$ are either (a) included in the payload of the failing block if the halt occurs at commit, or (b) recorded in the evidence bundle sealed at Ω for the next successful commit.

Definition 8.3 (Seed function and inputs)**.** Let Seed be a pure KDF constructed from BLAKE3-256 (e.g., HKDF context tag `"specchain/seed-v1"`). For a run with suite identifier `suite`, parent list $P = (p_1, \ldots, p_k)$ (already finalized and sorted lexicographically), and fingerprint `proof_surface`, define the per-run seed as

$$\mathsf{seed} \;:=\; \mathsf{Seed}\big(\texttt{root_print} \,\|\, \texttt{suite} \,\|\, p_1 \,\|\, \cdots \,\|\, p_k \,\|\, \texttt{proof_surface}\big),$$

computed inside `EnvLock`. Stage-local RNG streams derive from seed by domain-separated labels (e.g., `"EF"`, `"DRO"`, ...).

Theorem 8.4 (RNG/commit acyclicity)**.** *Assume Definitions 4.3 and 8.1 and the seed discipline of Definition 8.3. Then the combined evaluation + commit relation is acyclic: no commit can (directly or indirectly) influence a seed used earlier on a cycle.*

Proof. Let G be the directed graph on commits with an edge $C \to C'$ iff $\mathsf{H}(C) \in C'.\texttt{parent_hashes}$. By Definition 4.3, G is a DAG over finalized commits. For any commit C, its seed is

$$\mathsf{seed}(C) = \mathsf{Seed}\big(\texttt{root_print} \,\|\, \texttt{suite}(C) \,\|\, \underbrace{p_1 \| \cdots \| p_k}_{\text{parents of } C} \,\|\, \texttt{proof_surface}(C)\big),$$

a pure function of: (i) `root_print` (constant), (ii) `suite`(C) (run-local constant), (iii) finalized parent digests (strict ancestors in G), and (iv) `proof_surface`(C), which is computed at Ω *before* block assembly and hashing. Suppose toward contradiction there were a cycle $C_0 \to C_1 \to \cdots \to C_m = C_0$ in the evaluation+commit relation such that some $\mathsf{seed}(C_j)$ depends (transitively) on $\mathsf{H}(C_j)$. Then $\mathsf{H}(C_j)$ would need to appear among its own parents or in data derived from them, or else `proof_surface`(C_j) would need to depend on $\mathsf{H}(C_j)$. The former contradicts the parent rule (no self-reference) and the finalization precondition; the latter contradicts the evaluation order (`proof_surface` is finalized at Ω prior to block assembly) and the deterministic envelope (no post-commit feedback). Thus no such cycle exists. □

Proposition 8.5 (Safety-or-halt primitive)**.** *Under Definitions 8.1 and 8.2 and invariants **I6**–**I14** (Section 7), any deviation from canonical encoding, axis isolation, guard-first pricing, seed discipline, or DAG hygiene is detected and results in a deterministic halt with a `RejectionCert`. Hence any recorded commit is either produced by a replayable, policy-compliant evaluation or there is no commit at all.*

9 Evaluation Order

A canonical acyclic order for a single run is:

E1. EF — Ingest data; compute `"raw_data_digest"`; validate `"config_cert"`.

E2. Seed PRNG — Derive seed from (`root_print`, `suite`, `parent_hashes`, `proof_surface`) inside `EnvLock` (Definition 8.3).

E3. DRO — Construct layerwise objects; log FrontierRows; accumulate Δ_{fr} with guard-first enforcement (Definition 6.2).

E4. PR — Compute predicates; emit certificates (e.g., `ThetaCert`, `VCert`).

E5. CBF — Enforce budgets/filters; apply only *legal* retunes (Definition 3.1); emit `RetuneCerts`.

E6. Ω — Close gates; assemble the byte-exact evidence bundle; compute `proof_surface` from SurfaceV1 under `canon/v1` (Definition 5.1); insert into payload.

E7. Commit — Assemble header/payload (both `canon/v1`); hash and append to Ledger with only finalized parents (Section 4).

Proposition 9.1 (Order safety and replayability)**.** *Any execution that respects the partial order induced by commit-DAG edges and the sequential constraints **E1**–**E7** is safe and replayable.*

Proof sketch. Replayability follows from `canon/v1` and `proof_surface` binding (Definition 5.3). Safety follows from Loader invariants **I6**–**I14** (Section 7) and acyclicity of evaluation+commit (Definition 8.4); any violation yields a deterministic halt with a `RejectionCert`. □

10 Axis Policies and Quantale/Quantaloid Upgrade

Definition 10.1 (Axis policy)**.** For each axis $i \in I$ we declare a *policy* π_i that fixes:

(a) the carrier poset $(Q_i, \leq_i)$ with identity e_i and absorbing bottom $\bot_i$;

(b) a closed, monotone binary law $\otimes_i \in \{\text{cumulative, idempotent, multiplicative, superadditive}\}$, interpreted respectively as $+$, max, $\cdot$, and a declared superadditive $+$-like law;

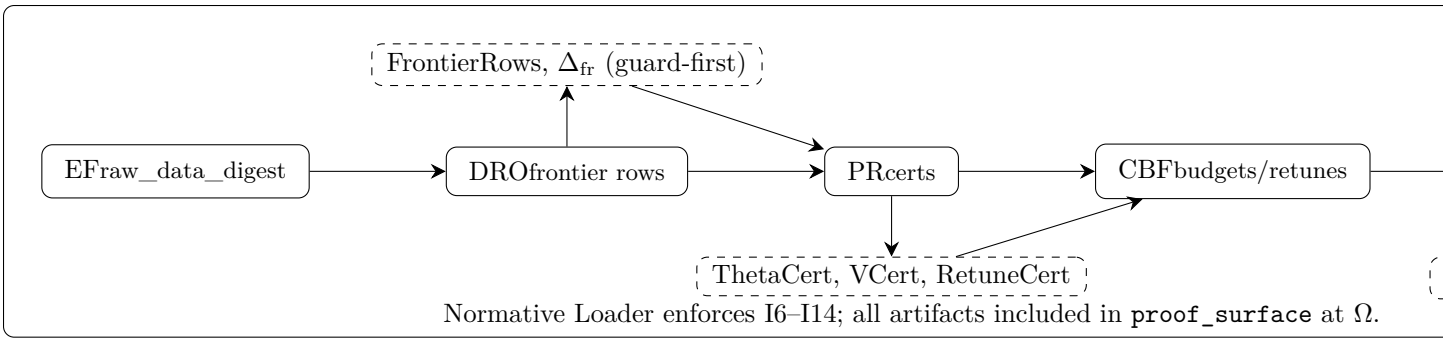


Figure 1: High-level adapter flow and artifacts. All artifacts are canonicalized (`canon/v1`) and bound into `proof_surface` at Ω.

Table 2: Canonical artifacts (brief). Detailed schemas appear later.

Artifact	**Description (all JSON under `canon/v1`)**
`RawDataDigest`	Digest of canonical raw input bytes (EF).
`ConfigCert`	Certificate binding configuration/policy for the run.
`FrontierRow`	Priced crossing with per-row constants; aggregates into Δ_{fr} after guard.
`ThetaCert`, `VCert`	Predicate/validation certificates (PR).
`RetuneCert`	Byte-level evidence of non-expansive retunes (CBF).
`RejectionCert`	Deterministic halt record (predicate id, witness hash, seed snapshot, parent tip).
`adapter_decls`	Canonical set of declared adapters (ids, versions, axes, policy digest).
`axis_types`	Axis policy catalog digest (drives $(\mathbf{Q}, \wedge)$; cf. Section 10).
`proof_surface`	Run fingerprint per Definition 5.1; included in payload at Ω.

(c) a flag $\mathsf{complete}_i \in \{\texttt{true}, \texttt{false}\}$ certifying that $(Q_i, \leq_i)$ is a complete lattice and that $\otimes_i$ distributes over arbitrary suprema when $\mathsf{complete}$ is `true`.

Write $I_c \subseteq I$ for the indices with $\mathsf{complete}_i = \texttt{true}$.

Definition 10.2 (Policy-driven typed product)**.** Given $\{\pi_i\}_{i \in I}$, the product $\mathbf{Q} = \prod_{i \in I} Q_i$ with product order and coordinatewise aggregation $\wedge$ (as in Definition 2.2) forms a *typed quantaloid* whose objects are the axes $i \in I$ and whose hom-objects are the monoids $(Q_i, \otimes_i, e_i)$. On the complete slice $\mathbf{Q}|_{I_c} = \prod_{i \in I_c} Q_i$ we obtain a *commutative unital quantale* with joins and products computed coordinatewise.

Proposition 10.3 (Distributivity-on-demand)**.** *If $i \in I_c$ and $\{x^\alpha\}_\alpha \subseteq Q_i$, then for all $y \in Q_i$,*

$$y \otimes_i \bigvee_\alpha x^\alpha \;=\; \bigvee_\alpha (y \otimes_i x^\alpha), \qquad \bigvee_\alpha x^\alpha \otimes_i y \;=\; \bigvee_\alpha (x^\alpha \otimes_i y).$$

No such requirement is imposed for $i \notin I_c$ beyond the monoid laws.

Proposition 10.4 (Adapter functoriality under policy refinement)**.** *Let $\pi \preceq \pi'$ denote* policy refinement *that preserves each axis's monoid $(Q_i, \otimes_i, e_i)$ and (optionally) upgrades* $\mathsf{complete}$ *from* `false` *to* `true` *or strengthens $\leq_i$. Any adapter A that is axis-isolating, monotone, and $(\mathbf{Q}, \wedge)$-homomorphic under π remains so under π'.*

Remark 10.5 (Backward compatibility)**.** The classic (S, M) discipline is recovered by policies with $\otimes_i{=}+$ and $e_i{=}0_i$ for $i \in S$, and $\otimes_i{=}\max$ and $e_i{=}-\infty_i$ for $i \in M$, with $\mathsf{complete} = \texttt{true}$ on max-axes. The absorber $\perp_i$ persists as a designated bottom that is absorbing for $\otimes_i$.

Normative JSON schema (axis policy catalog, `canon/v1`). A run carries an *axis policy catalog* whose canonical bytes contribute to `"axis_types"` in SurfaceV1 (Definition 5.1). The minimal shape is a list of per-axis objects:

- `"id"`: unique axis identifier (string; ASCII),
- `"units"`: human-readable units or `"dimensionless"`,
- `"compose_law"`: one of `"cumulative"`, `"idempotent"`, `"multiplicative"`, `"superadditive"`,
- `"complete"`: boolean,
- `"carrier"`: optional tag among `"scalar"`, `"interval"`, `"sub_gaussian"`, `"robust_set"` (used by $\mathbf{Q}'$; see Section 11).

Listing 4: Illustrative axis policy catalog (canon/v1).

```
[{"carrier":"scalar","complete":false,"compose_law":"cumulative","id":"
   energy","units":"joule"},
 {"carrier":"scalar","complete":true,"compose_law":"idempotent","id":"risk
    ","units":"dimensionless"}]
```

Proposition 10.6 (Well-formedness checks (Loader))**.** *The Loader validates, before evaluation:*

(A1) ***Law compatibility****: Declared $(Q_i, \otimes_i, e_i, \perp_i)$ satisfy monoid laws and $\perp_i$ is absorbing.*

(A2) ***Completeness claims****: If `"complete"=true`, a complete lattice structure is available and recorded (or referenced) for $(Q_i, \leq_i)$.*

(A3) ***Adapter alignment****: Every adapter that writes axis i cites a policy digest matching the catalog; otherwise reject with `RejectionCert`.*

Remark 10.7 (Mechanization surface)**.** Part 1 exposes a module `/algebra/QuantaleQ.lean` that (i) defines `AxisPolicy`, (ii) constructs $(\mathbf{Q}, \wedge)$ and its quantale slice from a catalog, and (iii) proves Definitions 10.3 and 10.4 from the catalog witnesses. This bridges policy JSON $\to$ algebra *by construction.*

11 Uncertainty-Aware Extension $\mathbf{Q}'$

Definition 11.1 (Per-axis uncertainty carriers)**.** Fix the axis policy catalog of Section 10. For each axis $i \in I$, choose a carrier family Q_i' that *lifts* Q_i to represent uncertainty while preserving the identity e_i and absorbing bottom $\perp_i$. Typical choices (all with their natural orders) include:

(a) **Intervals** $[a, b]$ with $a \leq b$ and containment order ($[a, b] \leq [a', b']$ iff $a \geq a'$ and $b \leq b'$); lift $+$, max, and $\cdot$ pointwise/Minkowski.

(b) **Sub-Gaussian envelopes** $\langle \mu, \sigma, \psi_2 \rangle$ with convolution-style $+$ on means and Orlicz norm update for dispersion; order by coordinatewise $\leq$.

(c) **Robust sets** $\mathcal{U} \subseteq \mathbb{R}^d$ (nonempty, convex, closed), ordered by reverse inclusion ($\mathcal{U} \leq \mathcal{V}$ iff $\mathcal{U} \supseteq \mathcal{V}$); aggregates via Minkowski sum or setwise max.

Write $\mathbf{Q}' := \prod_{i \in I} Q'_i$ with product order $\leq_{\mathbf{Q}'}$ and coordinatewise aggregation $\wedge$ obtained by lifting the per-axis laws $\otimes_i$ declared in π_i (cf. Definition 10.1).

Assumption 11.2 (Well-posedness of carriers)**.** For each axis i: (i) Q'_i contains e_i and $\bot_i$; (ii) $\otimes_i$ lifts to a closed, monotone binary operation on Q'_i with $\bot_i$ absorbing; (iii) if $\mathsf{complete}_i = \texttt{true}$ then $(Q'_i, \leq_i)$ admits arbitrary suprema and $\otimes_i$ distributes over them. These witnesses are validated by the Loader via the axis-policy catalog.

Definition 11.3 (Budgets and acceptance stances)**.** For each axis i fix a *budget* B_i and a stance $s_i \in \{\mathsf{prob}, \mathsf{robust}, \mathsf{hybrid}\}$. Given a measurable performance map g_i:

$$x_i \models_{s_i} B_i \iff \begin{cases} \Pr_\theta[g_i(x_i, \theta) \leq B_i] \;\geq\; 1 - \delta_i, & s_i = \mathsf{prob},\ \delta_i \in (0,1) \\ \sup_{u \in \mathcal{U}_i} g_i(x_i, u) \;\leq\; B_i, & s_i = \mathsf{robust} \\ \max\{\mathsf{prob}, \mathsf{robust}\} \text{ holds}, & s_i = \mathsf{hybrid} \end{cases}$$

and lift to $x = \langle x_i \rangle_i \in \mathbf{Q}'$ by coordinatewise conjunction: $x \models B \iff \forall i,\ x_i \models_{s_i} B_i$.

Assumption 11.4 (Closure, measurability, and budget monotonicity)**.** For each i: (i) acceptance regions $\{x_i : x_i \models_{s_i} B_i\}$ are *closed* sets in Q'_i; (ii) g_i is measurable in its random/uncertain argument; (iii) budget maps are monotone: if $x_i \leq_i y_i$ then the minimal budget that accepts y_i is $\geq$ the minimal budget that accepts x_i (no "free acceptance").

Theorem 11.5 (Monotonicity and adapter preservation)**.** *Under Definitions 11.2 and 11.4,* $(\mathbf{Q}', \wedge)$ *is monotone. If A is axis-isolating and monotone on inputs and its per-axis laws lift to Q'_i, then* acceptance is preserved*:*

$$x \models B \quad \Longrightarrow \quad A(x) \models B.$$

Proof sketch. Monotonicity of $\wedge$ is coordinatewise from the lifted $\otimes_i$. Closedness of acceptance regions ensures limits of improving sequences remain accepted. Axis isolation plus monotonicity of A and of budget maps yields $A(x)_i \leq_i A(y)_i$ whenever $x_i \leq_i y_i$, whence A cannot invalidate a previously satisfied coordinate. Conjunction over i completes the proof. □

Proposition 11.6 (Sound lifting of guards)**.** *If a frontier row's priced inputs on axis i are represented by an uncertainty element $x_i \in Q'_i$ and the stance is* robust *or* hybrid*, then the guard of Definition 6.2 must be checked against the* worst-case *(supremal) realization permitted by x_i; for* prob*, the guard is checked at the $(1 - \delta_i)$-quantile. Any $\bot_i$ forces rejection prior to pricing.*

Example 11.7 (Intervals + sub-Gaussian)**.** Let energy be a sum-axis with interval carrier and budget $B_{\text{eng}} = 100$, and risk a max-axis with sub-Gaussian carrier and stance prob with $\delta = 0.05$. For $x_{\text{eng}} = [48, 52]$ and $x_{\text{risk}} = \langle \mu = 0.2, \sigma = 0.05, \psi_2 \rangle$, $x \models B$ holds if $\sup x_{\text{eng}} \leq 100$ and the 95%-quantile of risk is $\leq B_{\text{risk}}$. Aggregating two such items uses Minkowski addition on energy intervals and the lifted max (idempotent join) on risk.

Normative JSON (uncertainty adornments, `canon/v1`). Axes that use $\mathbf{Q}'$ MAY include uncertainty fields in the policy catalog (Section 10): `"stance"` ∈ `"prob"|"robust"|"hybrid"`, optional `"delta"` for prob, and a `"robust_set"` digest or inline descriptor for robust. The Loader validates domain/range, closedness claims, and that adapter implementations match the declared carrier/laws.

Remark 11.8 (Mechanization surface)**.** Module `/algebra/UncertaintyQprime.lean` provides: (i) carrier interfaces, (ii) lifted operations and order, (iii) proofs of Definition 11.5 from carrier witnesses, and (iv) a guard-lifting lemma Definition 11.6.

12 Computational Complexity of Δ_{fr}

Definition 12.1 (Frontier Reassociation Minimization (optimization/decision))**.** Fix a policy π that specifies (i) legal *reassociations* (partitions of a multiset L of frontier rows into groups constrained by *op_type*/metadata), and (ii) a per-group kernel φ_π that maps each group $G \subseteq L$ to a contribution in $(\mathbf{Q}, \wedge)$, aggregated by $\wedge$ across groups. The *Frontier Reassociation Minimization (FRM)* problem is:

$$\text{FRM-Opt:} \qquad \inf_{T \in \mathsf{Assoc}_\pi(L)} \Delta_{\text{fr}}(T, \pi) \quad \text{where} \quad \Delta_{\text{fr}}(T, \pi) \;:=\; \bigwedge_{G \in T} \varphi_\pi(G).$$

The associated decision problem is:

$$\text{FRM-Dec:} \qquad \text{Given threshold } K \in \mathbf{Q}, \text{ does there exist } T \in \mathsf{Assoc}_\pi(L) \text{ s.t. } \Delta_{\text{fr}}(T, \pi) \leq_{\mathbf{Q}} K \;?$$

Here $\mathsf{Assoc}_\pi(L)$ is the set of legal partitions of L under π.

Theorem 12.2 (Strong NP-hardness of exact Δ_{fr})**.** *FRM-Dec is strongly NP-hard even under the following restrictions:*

(R1) a single *sum-axis $i^\star \in S$ (all other axes identities),*

(R2) legal groups are of exactly three *rows (arity-*3 *partition),*

(R3) the per-group kernel on $i^\star$ is the overflow penalty $\varphi_\pi(G)_{i^\star} = \max\{0, \sum_{r \in G} c(r) - T\}$ *for a fixed capacity $T > 0$ and nonnegative costs $c(r)$,*

(R4) the threshold is $K_{i^\star} = 0$ (and identities elsewhere).

*Proof sketch (reduction from 3-*Partition*, strong).* Given an instance of 3-Partition: multiset $S = \{s_1, \ldots, s_{3m}\}$ of positive integers with $\sum_i s_i = mT$ and bounds $T/4 < s_i < T/2$ (ensuring any feasible subset has size 3), construct L with one row r_i of cost $c(r_i) = s_i$ on axis $i^\star$. Let π enforce groups of exactly three rows; set φ_π as in (R3) and the decision threshold $K_{i^\star} = 0$. Any partition $T \in \mathsf{Assoc}_\pi(L)$ yields

$$\Delta_{\text{fr}}(T, \pi)_{i^\star} = \sum_{G \in T} \max\{0, \sum_{r \in G} s_r - T\}.$$

($\Rightarrow$) If the 3-Partition instance is feasible, there exists a partition into m triples each summing exactly to T, hence every group overflow is 0 and the aggregate equals $0 \leq K_{i^\star}$.

($\Leftarrow$) Conversely, suppose there is T with aggregate overflow 0. Because $\sum_i s_i = mT$, if any group sums to $< T$, another must exceed T (given the fixed arity and nonnegativity), producing strictly positive overflow—contradiction. Thus every group sums exactly to T, witnessing a valid 3-Partition. The reduction is polynomial, uses only one sum-axis, and relies on integers bounded by a polynomial in the input size; hence the hardness is *strong*. Membership in NP for FRM-Dec follows by certifying the partition T and evaluating $\Delta_{\text{fr}}(T, \pi)$ in polynomial time. □

Remark 12.3 (Scope and variants)**.** The hardness persists for piecewise-linear convex penalties, superadditive group costs, or when overflow is replaced by absolute-deviation penalties $\sum_G |\sum_{r \in G} c(r) - T|$. Hybrid settings with additional idempotent (max) axes remain NP-hard by projection onto the sum-axis. Detailed constructions and further variants are developed in Part 2.

13 Sound Upper-Bound Approximations

Exact FRM is strongly NP-hard (Definition 12.2), so we provide *sound* approximations that are safe to use inside the Loader: they never underestimate debit and therefore cannot weaken budgets.

Definition 13.1 (Approximation regimes)**.** For a legal log L and policy π, we define three families of solvers that return approximate debits in $(\mathbf{Q}, \wedge)$:

G1 — Greedy pairing.
A single pass that repeatedly merges the legal group (respecting π) with the smallest marginal increase in the current aggregate debit on $(\mathbf{Q}, \wedge)$, tie-broken by dominance on idempotent (max) axes. Greedy halts when no beneficial merge remains or a policy-imposed arity bound is hit. Denote its output by $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G1})}(L, \pi)$.

G2 — Bounded-width DP.
Let $G_\pi(L)$ be the association graph induced by π; fix a window parameter $w \in \mathbb{N}$. Perform a dynamic program that is *exact* on any cut of treewidth $\leq w$ and aggregates across windows with a monotone join. Denote the result by $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G2},w)}(L, \pi)$.

G3 — Convex/flow relaxations with monotone rounding.
Encode FRM-Dec as a 0–1 flow/assignment program, relax to LP/SOCP (or robust counterparts for $\mathbf{Q}'$), solve the relaxation, then apply a *monotone rounding* map round that (i) preserves feasibility and (ii) cannot *decrease* any coordinate of $(\mathbf{Q}, \wedge)$. The output is $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G3})}(L, \pi)$.

When π and L are understood we write $\widehat{\Delta}_{\mathrm{fr}}^{(\cdot)}(L)$.

Theorem 13.2 (Sound upper bounds)**.** *For any legal log L and policy π,*

$$\Delta_{\mathrm{fr}}(L, \pi) \;\leq_{\mathbf{Q}}\; \widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G1})}(L, \pi) \;\leq_{\mathbf{Q}}\; \widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G2},w)}(L, \pi) \;\leq_{\mathbf{Q}}\; \widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G3})}(L, \pi),$$

coordinatewise in $(\mathbf{Q}, \leq_{\mathbf{Q}})$. In particular, substituting any $\widehat{\Delta}_{\mathrm{fr}}^{(\cdot)}$ for Δ_{fr} in a budget check is safe*.*

Proof sketch. G1: Each greedy merge computes a marginal $\wedge$-increase that over-approximates any deferred, later merge; monotonicity of $\wedge$ (Definition 2.3) ensures no step can reduce a coordinate. G2: Exactness within a window followed by monotone aggregation across windows yields an upper bound by subadditivity (Definition 6.3 F3). G3: The relaxation objective/value lower-bounds the *combinatorial* optimum, but the rounding round is required to be coordinatewise monotone nondecreasing in $(\mathbf{Q}, \wedge)$, hence cannot return a debit smaller than the true optimum; composing these facts gives the bound. □

Proposition 13.3 (Tightness regimes)**.** *The approximations are* exact *in the following cases:*

*(T1) For **G2**, if the association graph $G_\pi(L)$ has treewidth $\leq w$, then $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G2},w)}(L) = \Delta_{\mathrm{fr}}(L)$.*

*(T2) For **G3**, if the relaxed polytope is totally unimodular (e.g., bipartite matching/assignment forms)* or *the priced axes are all idempotent and complete (Section 10), then the relaxation is integral and $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G3})}(L) = \Delta_{\mathrm{fr}}(L)$.*

Remark 13.4 (Choosing a regime)**.** G1 is $O(|L| \log |L|)$ and suitable for online gating. G2 trades accuracy for w-exponential but graph-sparse performance. G3 offers the strongest global bounds and supports $\mathbf{Q}'$ (uncertainty) via robust LP/SOCP, at the cost of a solver and rounding certificate.

Rounding and receipts (normative surface). For **G3**, the Loader requires a *rounding receipt* with:

- the relaxation solution value and constraint digest,
- a witness that round is axiswise monotone (identifier + code digest),

- the realized groups or flows and the resulting coordinates in $\mathbf{Q}$,
- a proof hash that $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G3})} \geq_{\mathbf{Q}} \Delta_{\mathrm{fr}}^{\mathrm{relax}}$.

The receipt's canonical bytes (under `canon/v1`) are included in the `"evidence_bundle"` bound by `proof_surface`.

Mechanization hooks. Module `/debit/DeltaFrBounds.lean` provides:

(B1) an abstract interface for G1/G2/G3 and proofs of Definition 13.2,

(B2) a formal statement of Definition 13.3 with witnesses (tree decompositions; TU certificates),

(B3) a checker that accepts a *rounding receipt* and verifies monotone rounding against the recorded relaxation instance.

Example 13.5 (Window-exactness)**.** Let $G_\pi(L)$ decompose into a chain of cliques of size $\leq w{+}1$. Then the bounded-width DP computes each clique exactly and joins the boundaries monotonically, hence $\widehat{\Delta}_{\mathrm{fr}}^{(\mathrm{G2},w)}(L) = \Delta_{\mathrm{fr}}(L)$ by Definition 13.3(T1).

14 Mechanization & Extraction Hooks

This section fixes the *mechanization surface* promised by Part 1. It is intentionally small and stable so downstream proofs and CI can treat Part 1 as an immutable contract.

Scope. All artifacts below are *definitions, statements, or checkers* that Part 1 *must* provide. Proofs deferred to later parts are not exposed here, except where Part 1 has stated axioms or lemmas already proved in-text.

Types, Classes, and Constructors (Lean surface)

- `AxisPolicy`: record mirroring the axis policy JSON (Section 10); constructors validate monoid laws, identity e_i, absorbing bottom $\bot_i$, and (when claimed) completeness witnesses.
- `Carrier`/`Monoid`/`CompleteLattice`/`Quantale`: minimal typeclasses to state and use Definition 10.3.
- `Q_Product`: builder that takes a catalog of `AxisPolicy` and constructs $(\mathbf{Q}, \wedge)$ and its complete slice; exposes theorems of Definitions 2.3, 10.3 and 10.4.
- `Qp_Lift`: interfaces for interval/sub-Gaussian/robust-set carriers with lifted operations (Section 11).

Axioms/Witnesses (as exported theorems)

- **Frontier axioms** (Definition 6.3): exported as named theorems `DeltaFr.id`, `DeltaFr.nonneg`, `DeltaFr.subadd`, `DeltaFr.mono`, `DeltaFr.dimensionless`.
- **Adapter functoriality** (Definitions 2.7 and 10.4).
- **Acceptance preservation** for $\mathbf{Q}'$ (Definition 11.5) under recorded carrier witnesses.
- **Acyclicity primitive** (Definition 8.4) exported as a lemma the Loader can depend on.

Table 3: Mechanization modules guaranteed by Part 1.

Module	Content (statements/checkers; `canon/v1` IO)
`/algebra/QuantaleQ.lean`	`AxisPolicy`, product builder for $(\mathbf{Q}, \wedge)$, proofs of Definitions 2.3, 10.3 and 10.4.
`/algebra/UncertaintyQprime.lean`	Carriers (interval/sub-Gaussian/robust-set) and lifted operations; proof of Definition 11.5; guard lifting lemma.
`/debit/DeltaFrAxioms.lean`	Interfaces and named theorems for Definition 6.3.
`/debit/DeltaFrHardness.lean`	Formalization of FRM-Dec and statement of Definition 12.2.
`/debit/DeltaFrBounds.lean`	Abstract APIs for G1/G2/G3; proofs of Definitions 13.2 and 13.3; rounding-receipt checker.
`/checkers/Guards.lean`	(C1) Frontier guard checker over canon-bytes; rejects on $\perp$-priced inputs.
`/checkers/SubAddSpot.lean`	(C2) Spot-checker for subadditivity with counterexample emission.
`/checkers/SurfaceBind.lean`	(C3) `proof_surface` re-encoder/binder over `canon/v1`.

Extracted Checkers (C1–C3, receipts)

(C1) **Frontier guard checker** (Definition 6.2): validates $\kappa \le C_{\text{salt}}(1 + C_I)$ and rejects rows with $\perp$ on any priced input *before* aggregation. Input/outputs are canon-bytes (`canon/v1`).

(C2) **Subadditivity spot-checker**: draws pairs (L_1, L_2) of compatible prefixes from a log and refutes violations of `DeltaFr.subadd` with a concrete counterexample (canon-bytes).

(C3) **`proof_surface` binder**: recomputes $\mathsf{SurfaceV1}_{\texttt{canon/v1}}$ and $H(\cdot)$; checks byte equality to the recorded `proof_surface` (Definition 5.1).

For **G3** approximations (Section 13), an additional *rounding receipt checker* verifies: (i) relaxation instance digest, (ii) solution value, (iii) that `round` is axiswise monotone (by code digest and replay), and (iv) that the rounded debit is $\geq_{\mathbf{Q}}$ the relaxed value.

Module Layout (normative)

CI Expectations

- **Canon discipline**: all checker inputs/outputs are `canon/v1`-encoded; non-canonical JSON is rejected with a `RejectionCert`.
- **Replay seeds**: CI replays runs using `seed` (Definition 8.3); any seed drift fails the job.
- **Negative cases**: include disturbed key order, whitespace injection, non-finite numbers, and non-minimal numeric forms for `proof_surface` and certificates (expected to fail binding).
- **Unknowns**: tests ensure $\perp$ cannot be cancelled by aggregation and triggers guard-time rejection when priced.

Remark 14.1 (Minimalism principle)**.** No additional axioms are introduced beyond those named in this section. Any future strengthening must occur via policy refinement (Definition 10.4) or new modules in later parts, without altering the contractual surface of Part 1.

15 Cross-Cutting Invariants (Do/Don't)

The following invariants are *global* and apply across all stages, artifacts, and policies. They summarize the safety posture fixed by Part 1 and are enforced by the Loader (Section 7) and by canonical binding (Sections 4 and 5).

Do — required behaviors.

- **Replayability & canon discipline.** Every JSON artifact that influences priced outputs or evidence is encoded under `canon/v1` and bound either directly by `proof_surface` or indirectly via payload hashing (Definitions 4.2 and 5.1).
- **Monotonicity of adapters.** All adapters are axis-isolating and monotone into $(\mathbf{Q}, \wedge)$; composition preserves monotonicity (Definitions 2.7 and 7.4).
- **Non-expansive retunes only.** Retunes occur only at **CBF**, satisfy $\Sigma' \preceq \Sigma$ (Definition 3.1), and emit a `RetuneCert`.
- **Sound approximations.** When exact Δ_{fr} is infeasible, use only $\widehat{\Delta}_{\text{fr}}$ from G1/G2/G3 (Section 13); these are *upper bounds* (Definition 13.2).
- **Unknowns are fail-closed.** Any $\bot$ in a priced input forces guard-time rejection (Definition 6.2); $\bot$ that enters $\mathbf{Q}$ is absorbing and cannot be cancelled (Definition 2.3, Definition 6.4).
- **Seed discipline & DAG hygiene.** Seeds derive only from (`root_print`, `suite`, `parent_hashes`, `proof_surface`) inside `EnvLock`; commits reference only finalized parents (Definitions 4.3, 8.3 and 8.4).
- **Evidence closure at Ω.** Close and digest the evidence bundle at Ω and compute `proof_surface` before commit (Section 9).
- **Deterministic halt on violation.** Any invariant breach yields a `RejectionCert` with canonical bytes (Definition 8.2).

Don't — prohibited behaviors.

- **No non-canonical IO.** Do not accept or emit JSON with unsorted keys, whitespace variation, non-finite numbers, or non-minimal numeric forms (Definition 4.1).
- **No axis bleed.** Adapters must not write to undeclared axes; such writes are violations of I6 (Definition 7.2).
- **No post-hoc normalization.** Never "fix up" bytes after they have contributed to a digest included in `proof_surface` or a certificate (Definition 5.3).
- **No lower bounds for gating.** Do not substitute any *under*-approximation of Δ_{fr} in budget checks; only the sound $\widehat{\Delta}_{\text{fr}}$ regimes are permitted (Definition 13.2).
- **No ambient nondeterminism.** Wall-clock time, OS entropy, nondeterministic kernels, or locale/encoding differences must not influence priced outputs (Definition 8.1).
- **No bypass of guards/budgets.** Global heuristics, caches, or out-of-band flows may not bypass frontier guards or budget enforcement (Definition 7.3).

Proposition 15.1 (Safety summary)**.** *Under the above invariants together with **E1–E7** (Section 9), every run either (i) produces a replayable block whose `proof_surface` and payload bytes witness policy compliance, or (ii) halts deterministically with a canonical `RejectionCert`. There is no third case.*

16 Acceptance & CI Suite

This section fixes the *release-gating* checks that Part 1 requires. A build is *acceptable* only if **all** items below pass on the reference corpus and synthetic generators. All IO is canonical (`canon/v1`); any non-canonical input must be rejected with a `RejectionCert`.

Library/Proof Surface

- **A1 — Algebra compiles.** The `/algebra` and `/debit` modules build with the stated theorems available: `DeltaFr.id`, `DeltaFr.nonneg`, `DeltaFr.subadd`, `DeltaFr.mono`, `DeltaFr.dimensionless`, `Quantale.distrib_on_demand` (Definition 10.3), `Policy.refine_preserves` (Definition 10.4), and `Qp.accept_preserved` (Definition 11.5).
- **A2 — Acyclicity lemma usable.** `rng_acyclic` (Definition 8.4) is exported and consumed by the Loader unit tests.

Canonicalization and Binding

- **C1 — Canon-only JSON.** Fails on unsorted keys, whitespace, NaN/Inf, or non-minimal numeric forms (Definition 4.1).
- **C2 — Surface binder.** Re-encodes SurfaceV1 bytes and matches `proof_surface` exactly (Definitions 5.1 and 5.3); includes negative cases with single-bit perturbations.
- **C3 — Payload hashing.** Header/payload bytes (`canon/v1`) recompute the block digest; mismatches trigger `RejectionCert`.

Seeds and Replayability

- **S1 — Seed determinism.** Given fixed (`root_print`, `suite`, `parent_hashes`, `proof_surface`), the derived seed (Definition 8.3) is bit-stable across platforms in `EnvLock`.
- **S2 — No feedback cycles.** Model checks show no schedule that respects **E1–E7** (Section 9) permits a seed to depend on a future commit.

Frontier Pricing and Guards

- **F1 — Guard validator (C1).** Rows pass iff $\kappa \le C_{\text{salt}}(1 + C_I)$ and contain no $\bot$ in priced inputs (Definition 6.2); failures emit `RejectionCert`.
- **F2 — Subadditivity spot-checker (C2).** Random compatible splits L_1, L_2 satisfy $\Delta_{\text{fr}}(L_1 \uplus L_2) \le_{\mathbf{Q}} \Delta_{\text{fr}}(L_1) \wedge \Delta_{\text{fr}}(L_2)$ (Definition 6.3).
- **F3 — Unknowns are absorbing.** Any $\bot$ entering $\mathbf{Q}$ remains on that axis after any aggregation; no cancellation is possible (Definition 2.3, Definition 6.4).
- **F4 — Dimensionless invariance.** Unit rescalings in evidence do not change Δ_{fr} coordinates (axiom Definition 6.3).

Table 4: Minimum CI jobs required for a release.

Job ID	Stage	Pass condition
`build:lean`	Library	All Part 1 modules compile; theorems exported (A1,A2).
`test:canon`	Canon	C1–C3 pass; negative corpus rejected with `RejectionCert`.
`test:seed`	Loader	S1–S2 pass across Linux/OSX runners in `EnvLock`.
`test:frontier`	DRO/PR	F1–F4 pass on synthetic and recorded logs.
`test:approx`	CBF	G1–G3 bounds hold; tightness suites pass where applicable.
`test:policy`	Init	P1–P2 pass for all declared axes and carriers.

Approximations and Receipts

- **G1 — Upper-bounds only.** For each solver, verify $\Delta_{\mathrm{fr}} \leq_{\mathbf{Q}} \widehat{\Delta}_{\mathrm{fr}}^{(\cdot)}$ on all cases (Definition 13.2).
- **G2 — Tightness smoke tests.** Window-exact instances (treewidth $\leq w$) and TU polytopes yield equality as per Definition 13.3.
- **G3 — Rounding receipt.** The receipt contains relaxation digest, solution value, rounding code digest, and a replayed certificate that rounded debit $\geq_{\mathbf{Q}}$ relaxed value; Loader verifies before accept.

Axis Policies and Q

- **P1 — Catalog well-formedness.** Policies satisfy monoid/absorber laws; completeness claims include lattice witnesses (Definition 10.6).
- **P2 — Acceptance preservation.** For $\mathbf{Q}'$ carriers, adapters preserve acceptance on the test corpus (Definition 11.5); guards lift per Definition 11.6.

CI Matrix (normative jobs)

Remark 16.1 (Release rule)**.** A release is blocked unless all matrix jobs in Table 4 pass. Any failure must produce a canonical `RejectionCert` (or CI artifact) with a predicate id and witness hash suitable for audit.

Acknowledgments and Cross-Part Roadmap

Acknowledgments. We thank the SpecChain Working Group, external reviewers, and CI maintainers for feedback on algebraic interfaces, canonicalization pitfalls, and Loader edge cases. The present revision benefited from red-team exercises that uncovered unknown-propagation bugs now resolved by the fail-closed $\perp$ semantics.

Roadmap across the series.

Part 2 — Pricing Kernels & Ontologies.
Gives the *construction* of Δ_{fr} from per-frontier kernels; proves the axioms of Definition 6.3; supplies the risk/value ontology and a catalog of axis policies with worked examples; develops FRM variants and detailed reductions extending Section 12; and presents dimensionless normalization recipes used by pricing.

Part 3 — Normative Loader & Runtime Assurance.
Formalizes the Loader automaton and invariants I6–I14 (Section 7); mechanizes Definition 8.4 in a small-step LTS and derives the Safety-or-Halt theorem from Definitions 9.1 and 15.1; instantiates the seed KDF (Definition 8.3); and fixes the canonical schemas and state transitions for `RetuneCert` and `RejectionCert`.

Part 4 — Governance & Economics.
Specifies the on-/off-ledger governance protocol, including `GovernorUpdateCert`/`GovernorPerfCert`, adapter lifecycle and deprecation, policy refinement workflows (Definition 10.4), and economic controls for budget sufficiency and audit funding.

Part 5 — Safety Case & Pilot Oversight.
Assembles the end-to-end assurance case: STRIDE/TCB analysis, UI/UX normalization rules mapping user-facing unknowns to algebraic $\perp$ (Definition 2.4), operational playbooks, and field reports from pilot deployments. Includes red-team findings and mitigations.

Versioning note. Part 1 is a normative contract: later parts may extend policies and provide proofs, but *must not* weaken the invariants, canonicalization rules, or certificate surfaces fixed here.

A Axioms & Conventions

The items below are *normative*. They summarize global choices and hygiene rules fixed by Part 1; later parts may extend but not weaken them.

- **Terminology & symbols.** The canonical run-fingerprint field is `proof_surface` (JSON key `"proof_surface"`); the genesis constant is `root_print`. The terminal stage is Ω (capital Omega). The obsolete label `"proof_print"` must not appear in new outputs.
- **Canonicalization (`canon/v1`).** All JSON that influences priced outputs, seeds, or audit is encoded under `canon/v1`: UTF-8; *sorted* keys; *no* inter-token whitespace; numbers are *finite IEEE-754 doubles* serialized in *minimal decimal* form (no `+`, no superfluous zeros, no exponent when avoidable); strings escaped per JSON. Non-canonical inputs are rejected upstream.
- **Unknowns are fail-closed.** Each axis i has an absorbing bottom $\perp_i$ that is also the order bottom: $x \otimes_i \perp_i = \perp_i \otimes_i x = \perp_i$ and $\perp_i \leq_i x$. Any priced input that is unknown *forces* guard-time rejection; if $\perp$ enters $\mathbf{Q}$ it is strictly ledgered and cannot be cancelled by aggregation (Definitions 2.3 and 6.2).
- **Axis isolation (I6).** Adapters write only to their declared axes I_S; writes outside I_S are violations detected by the Loader and produce a `RejectionCert` (Definition 7.2).
- **Determinism & seeds.** All computation occurs inside (`EnvLock`, `ReplayPlan`); floating point, RNGs, serialization, and code/proof digests are pinned (Definition 8.1). Seeds derive only from (`root_print`, `suite`, `parent_hashes`, `proof_surface`) (Definition 8.3); no wall-clock or nondeterministic OS entropy may influence priced outputs.
- **Ledger hygiene.** Headers/payloads are `canon/v1`-encoded; blocks hash as $\mathsf{H}(b) = \mathsf{BLAKE3\text{-}256}(hbytes \| pbytes)$ (Definition 4.2). Commits reference only *finalized* parents; the induced Ledger is a DAG (Definition 4.3).

- **Evidence closure & binding.** At Ω the evidence bundle is closed and digested; `proof_surface` is computed from SurfaceV1 and inserted into the payload *before* commit (Section 9). Any modification after Ω is a violation.
- **Retunes are non-expansive.** Retunes occur only at **CBF** and must satisfy $\Sigma' \preceq \Sigma$ under the fixed rank map $\mu : \Sigma \to \mathbb{N} \times \mathbb{N}$ (lex order); each legal retune emits a `RetuneCert` (Definitions 3.1 and 3.2).
- **Dimension hygiene.** Frontier prices contributing to Δ_{fr} are *dimensionless* by normalization; unit rescalings do not change coordinates (Definition 6.3).
- **Fail-closed halts.** Any invariant violation (canon, guards, axis isolation, seed discipline, DAG hygiene) deterministically halts with a canonical `RejectionCert` whose bytes are bound by the ledger (Definition 8.2).